Swarm Logistics & Warehouse Automation
SDD Document

Jamal Tannous 208912337

Dan Ydov 039992391

Basel Mansour 208204859

Swarm Logistics & Warehouse Automation
SDD Document

1. Agent Internal Architecture
1.1. Architecture Overview

The system uses a **hybrid deliberative–reactive agent architecture** for each warehouse robot agent. This architecture is suitable because the agent must perform planned behavior, such as task selection and path planning, while also reacting immediately to local events such as blocked cells, nearby agents, congestion, and potential collisions.

The deliberative part is responsible for reasoning about tasks, goals, path planning, bidding, and long-term movement decisions. The reactive part handles immediate safety and coordination decisions at each simulation step. This matches the project definition from the SRS, where agents operate in a discrete grid-based warehouse, act synchronously, avoid collisions, participate in decentralized task allocation, and dynamically replan when conflicts occur.

The selected architecture also follows the MAS lecture definition of intelligent agents as autonomous, goal-oriented, adaptable decision-making entities.

1.2. Agent Reasoning Model

Each robot agent follows a simplified **BDI model**, where the agent maintains:

Beliefs
Represent the agent's current knowledge about itself, the environment, tasks, and other agents.

Examples of beliefs:

- Current position of the agent.
- Static warehouse map, including obstacles, pickup locations, and drop-off locations.
- Current assigned task, if one exists.
- Available tasks in the task pool.
- Estimated positions and short-term movement intentions of nearby agents.
- Reserved cells in the reservation table.
- Local congestion level around the agent.
- Current path and next planned movement.
- Deadlock or blocking status.

The SRS already defines that each agent knows its own position, the static environment layout, its assigned task, nearby agents' movements, and available tasks if globally visible.

Desires / Goals
Represent what the agent wants to achieve.

Main goals:

- Complete assigned pickup and delivery tasks.
- Minimize task completion time.
- Avoid collisions.
- Avoid contributing to congestion.
- Avoid and resolve deadlocks.
- Stay productive when tasks are available.
- Support the global system objective of maximizing throughput.

These goals are directly aligned with the agent goals defined in the SRS.

Intentions / Plans
Are the current committed plans selected by the agent.

Examples of intentions:

- Bid for a specific task.
- Navigate to a pickup location.
- Pick up an item.
- Navigate to a drop-off location.
- Deliver the item.
- Wait temporarily to avoid a conflict.
- Replan path due to congestion or blockage.
- Release reservations after completing movement.
- Participate in deadlock resolution.

1.3. Internal Agent Modules

Each robot agent is divided into the following internal modules:

1.3.1. Perception Module

The perception module updates the agent's beliefs at every simulation time step.

Inputs:

- Current grid state.
- Agent position.
- Task pool state.
- Nearby agent positions.
- Received communication messages.
- Reservation table updates.
- Congestion information.

Output:

- Updated belief state.

This module allows the agent to operate under partial observability, where it does not know future tasks and may only have local or communicated knowledge about other agents.

1.3.2. Task Reasoning Module

The task reasoning module decides whether the agent should participate in task allocation.

If the agent is idle and tasks are available, it evaluates candidate tasks and computes a bid value. The bid considers:

- Distance from current position to pickup location.
- Estimated distance from pickup to drop-off.
- Current workload.
- Estimated congestion along the route.
- Expected delay due to reservations or conflicts.

A lower bid value means the task is more suitable for the agent.

Example bid function:

$$BidCost_i(t) = \alpha \cdot Distance_i(t) + \beta \cdot Congestion_i(t) + \gamma \cdot Workload_i$$

Where:

- $Distance_i(t)$is the estimated travel distance for agent $i$to complete task t.
- $Congestion_i(t)$is the predicted congestion along the path.
- $Workload_i$represents whether the agent is idle or already busy.
- $\alpha, \beta, \gamma$are weights used to balance the cost components.

This matches the One Pager strategy of congestion-aware auction-based task allocation, where agents bid using distance, congestion, and workload.

1.3.3 Planning Module

The planning module generates a valid path for the agent.

Primary planner:

- **Cooperative A*** with a reservation table.

The planner searches in space and time, not only in space. That means the agent plans movements while considering which cells are reserved by other agents at future time steps.

The planner must avoid:

- Obstacles.
- Grid boundaries.
- Occupied cells.
- Future reserved cells.
- Edge conflicts, where two agents swap positions in the same time step.

Fallback planner:

- Independent A* with priority-based conflict resolution.

The fallback is used if cooperative planning becomes too expensive or if the system needs a simpler recovery mechanism.

1.3.4 Reactive Safety Module

The reactive safety module can override the current plan when immediate risk is detected.

Reactive rules include:

- If the next cell is occupied, wait or replan.
- If another agent intends to move into the same cell, resolve using priority.
- If congestion exceeds a threshold, choose an alternative path if possible.
- If the agent has waited too long, trigger deadlock detection.
- If a collision risk is detected, stop movement for the current step.

This module is important because a pure planner is not enough in a dynamic multi-agent environment. Without reactive behavior, the design would be fragile.

1.3.5 Communication Module

The communication module sends and receives coordination messages.

Message types include:

- Task announcement.
- Bid submission.
- Task award.
- Movement intention broadcast.
- Cell reservation request/update.
- Conflict warning.
- Deadlock notification.
- Replanning notification.

1.3.6 Execution Module

The execution module applies the selected action in the simulation.

Possible actions:

- Move up.
- Move down.
- Move left.
- Move right.
- Wait.
- Pick up item.
- Deliver item.
- Send message.
- Submit bid.
- Replan path.

1.4 Agent Decision Cycle

At every simulation time step, each agent executes the following decision cycle:

1. **Perceive** the current environment state.
2. **Update beliefs** about the grid, tasks, nearby agents, reservations, and congestion.
3. **Check current goal**:
    - If idle, search for available tasks.
    - If assigned, continue task execution.
    - If blocked, trigger conflict handling.
4. **Participate in auction** if unassigned tasks are available.
5. **Plan or update path** using Cooperative A*.
6. **Broadcast movement intention** for the next step.
7. **Check for conflicts** using received intentions and reservations.
8. **Apply reactive safety rule** if needed.
9. **Execute one action**.
10. **Update task state and logs**.

This cycle fits the SRS temporal model, where agents perceive, compute decisions, and execute actions synchronously at each discrete time step.

1.5 Agent State Machine

Each agent can be in one of the following states:

State	Description
IDLE	Agent has no assigned tasks and waits for available tasks.
BIDDING	Agent evaluates available tasks and submits bids.
ASSIGNED	Agent has received a task but has not yet reached pickup.
MOVING_TO_PICKUP	Agent is navigating to the pickup location.
PICKING_UP	Agent performs pickup action.
MOVING_TO_DROPOFF	Agent is navigating to the drop-off location.
DELIVERING	Agent performs delivery action.
WAITING	Agent waits due to conflict, reservation, or congestion.
REPLANNING	Agent computes a new path due to blockage or conflict.
DEADLOCK_RECOVERY	Agent participates in deadlock resolution.

State transitions:

- IDLE → BIDDING when tasks are available.
- BIDDING → ASSIGNED when the agent wins an auction.
- ASSIGNED → MOVING_TO_PICKUP after path generation.
- MOVING_TO_PICKUP → PICKING_UP when pickup location is reached.
- PICKING_UP → MOVING_TO_DROPOFF after item pickup.
- MOVING_TO_DROPOFF → DELIVERING when drop-off location is reached.
- DELIVERING → IDLE after task completion.
- Any movement state → WAITING if the next step is blocked.
- WAITING → REPLANNING if waiting exceeds a threshold.
- REPLANNING → DEADLOCK_RECOVERY if cyclic waiting is detected.

1.6 Why This Architecture Is Suitable

This architecture is suitable because the warehouse robots are not simple reflex agents. They must make planned decisions, communicate, compete/cooperate for tasks, avoid future conflicts, and react to local problems.

A purely reactive agent would be too weak because it would not plan efficient paths or reason about task allocation.

A purely deliberative agent would also be weak because it may react too slowly to dynamic congestion and local conflicts.

Therefore, the best design is a **hybrid BDI + reactive architecture**, where:

- BDI handles task reasoning, goal selection, and planning.
- Reactive rules handle safety, local conflicts, and immediate replanning.
- Communication supports decentralized coordination.
- Reservation-based planning reduces collisions and deadlocks.

2. Interaction & Communication Design

2.1 Communication Design Overview

The system uses a **decentralized agent communication model**. There is no central controller that assigns tasks or controls movement. Instead, warehouse robot agents exchange messages to coordinate task allocation, movement intentions, path reservations, conflict handling, and deadlock recovery.

Communication is needed for three main reasons:

1. **Task allocation** — agents negotiate which robot should handle each task.
2. **Movement coordination** — agents share short-term movement intentions to avoid collisions.
3. **Conflict and deadlock resolution** — agents notify each other when blocking situations occur.

This matches the SRS, which states that agents can communicate short-term intent information, planned movements, and cell reservations to coordinate actions and reduce conflicts.

The communication model assumes that messages are reliable and have negligible delays, as defined in the SRS. This is reasonable for the simulation stage, but in a real physical warehouse it would need additional handling for message loss, latency, and network failures.

2.2 Communication Protocol

The system uses a simplified **FIPA-ACL inspired protocol**. FIPA-ACL is suitable because it gives each message a clear communicative purpose, such as informing, requesting, proposing, accepting, or rejecting.

Each message contains the following fields:

Field	Description
message_id	Unique identifier of the message.
sender_id	ID of the sending agent.
receiver_id	ID of the receiving agent, or ALL for broadcast.
performative	The communication act, such as INFORM, CFP, PROPOSE, ACCEPT_PROPOSAL, REJECT_PROPOSAL, or REQUEST.
content	Message payload, such as task details, bid value, movement intention, or conflict warning.
timestamp	Simulation time step when the message was sent.
conversation_id	ID used to group related messages, such as all messages belonging to the same auction.

Example message structure:

⟨/⟩ *JSON*

```
{
 "message_id": "msg_105",
 "sender_id": "agent_3",
 "receiver_id": "ALL",
 "performative": "INFORM",
 "content": {
  "type": "movement_intent",
  "current_cell": [4, 7],
  "next_cell": [4, 8],
  "time_step": 22
 },
 "timestamp": 21,
 "conversation_id": "intent_broadcast_21"
}
```

2.3 Main Message Types

Message Type	Performative	Sender	Receiver	Purpose
TASK_ANNOUNCEMENT	CFP	Task pool / announcing agent	All idle agents	Announces that a new task is available.
TASK_BID	PROPOSE	Robot agent	Task announcer / all agents	Sends bid cost for a task.
TASK_AWARD	ACCEPT_PROPOSAL	Task announcer / auction winner resolver	Winning agent	Assigns the task to the best bidder.
TASK_REJECT	REJECT_PROPOSAL	Task announcer / resolver	Losing agents	Informs agents that their bid was not selected.
MOVEMENT_INTENT	INFORM	Robot agent	Nearby agents / all agents	Announces the next intended cell.
CELL_RESERVATION	REQUEST or INFORM	Robot agent	Other agents / reservation table	Reserves a cell for a future time step.
CONFLICT_WARNING	INFORM	Robot agent	Conflicting agent	Warns that two agents intend to occupy the same cell.
REPLAN_NOTICE	INFORM	Robot agent	Nearby agents	Announces that the agent is replanning.
DEADLOCK_ALERT	INFORM	Robot agent / detector	Involved agents	Indicates that a cyclic waiting condition was detected.
DEADLOCK_RESOLUTION	REQUEST	Selected resolving agent	Involved agents	Requests waiting, priority change, or path release.

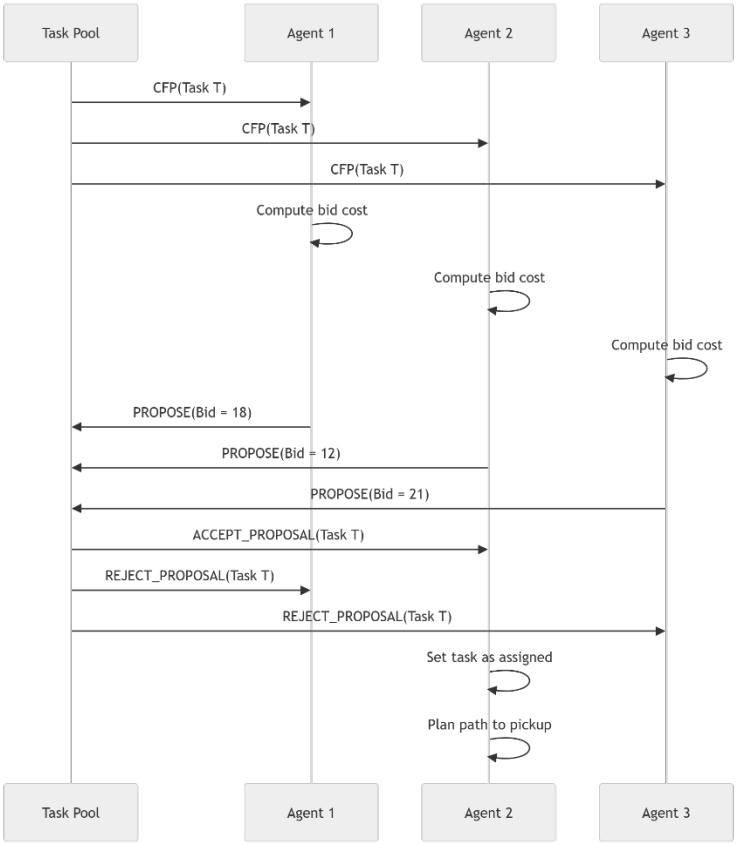
2.4 Task Auction Protocol

When a new task appears, agents use a **congestion-aware auction mechanism**.
The auction follows this logic:

1. A new task is added to the task pool.
2. The task is announced using a CFP message.
3. Idle agents calculate a bid cost.
4. Each participating agent sends a PROPOSE message containing its bid.
5. The lowest-cost valid bid wins.
6. The winning agent receives an ACCEPT_PROPOSAL.
7. Other agents receive REJECT_PROPOSAL.
8. The task state changes from unassigned to assigned.
9. The winning agent starts planning a path to the pickup location.

The bid is based on distance, congestion, and workload, which matches the One Pager's proposed congestion-aware auction-based task allocation strategy.

Task Auction Sequence Diagram



2.5 Movement Intention Protocol

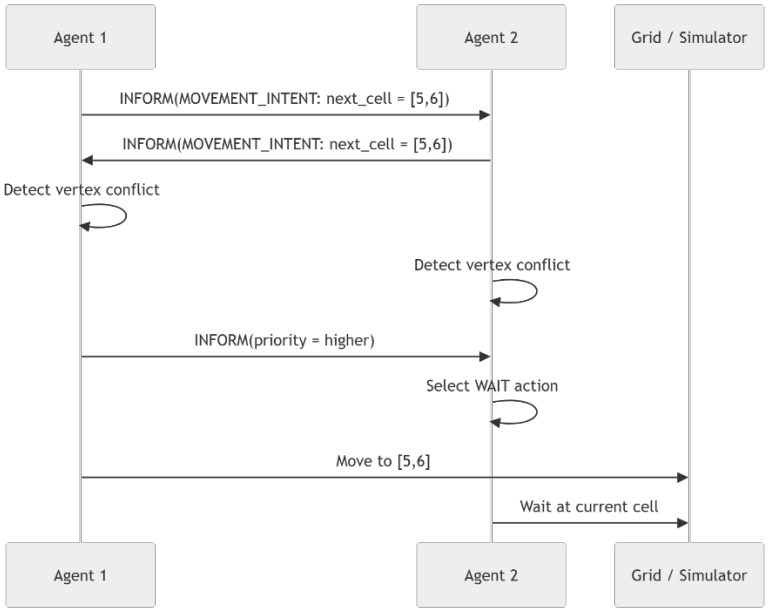
Before moving, each agent broadcasts its intended next cell. This reduces collision risk because agents can detect conflicts before executing the action.

The movement intention protocol follows this logic:

1. Agent computes its next planned cell.
2. Agent broadcasts MOVEMENT_INTENT.
3. Nearby agents compare received intentions with their own next move.
4. If no conflict exists, the agent moves.
5. If a conflict exists, agents apply priority rules.
6. Lower-priority agent waits or replans.

Conflict examples:

Conflict Type	Description	Resolution
Vertex conflict	Two agents want to enter the same cell at the same time.	Higher-priority agent moves; lower-priority agent waits/replans.
Edge conflict	Two agents want to swap positions in the same time step.	Swap is blocked; one agent waits.
Occupancy conflict	Agent wants to move into a currently occupied cell.	Agent waits or replans.



2.6 Reservation-Based Path Coordination

For cooperative path planning, agents use a **reservation table**. The reservation table stores which cells are reserved by which agents at future time steps.

Each reservation has the following structure:

Field	Description
agent_id	The agent that reserved the cell.
cell	Reserved grid cell.
time_step	Future time step for the reservation.
reservation_type	Vertex reservation or edge reservation.
expires_at	Time step after which the reservation is cleared.

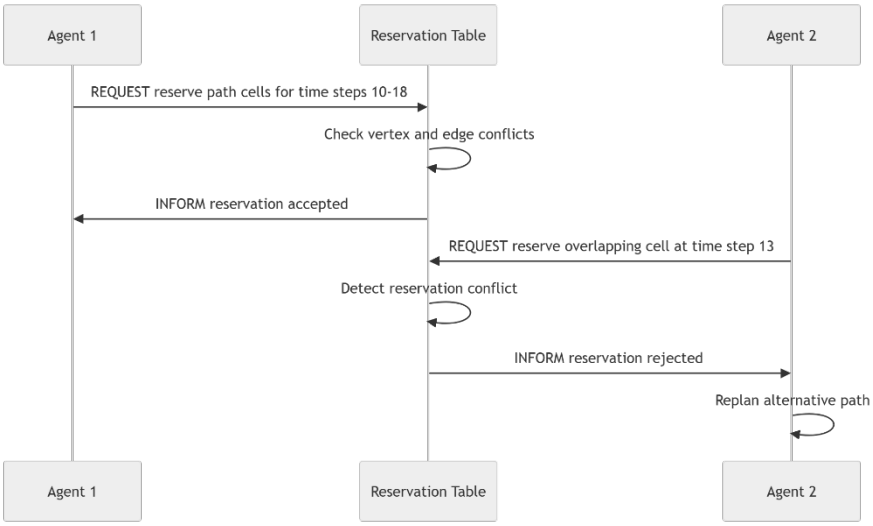
Example:

⟨/⟩ JSON

```
{
 "agent_id": "agent_2",
 "cell": [6, 4],
 "time_step": 15,
 "reservation_type": "vertex",
 "expires_at": 16
}
```

The reservation table supports the Cooperative A* planner by preventing agents from planning paths through the same cell at the same time. This directly supports the project's primary planning approach: Cooperative A* with time-space reservations.

Reservation Sequence Diagram



2.7 Conflict Resolution Protocol

When two or more agents detect a movement conflict, they use a priority-based conflict resolution mechanism.
Priority can be calculated using:

1. Agent with an active delivery task has priority over idle agents.
2. Agents closer to task completion has priority.
3. Agent that has waited longer has priority.
4. If still tied, lower agent ID wins as deterministic tiebreaker.

The conflict resolution protocol:

1. Agents exchange movement intentions.
2. Conflict is detected.
3. Agents compare priority values.
4. Higher-priority agent keeps its movement.
5. Lower-priority agent waits or replans.
6. If the same conflict repeats for several steps, deadlock detection is triggered.

This is necessary because the system acts synchronously, meaning all agents choose and execute actions at each discrete time step.

2.8 Deadlock Communication Protocol

A deadlock occurs when a group of agents are waiting for each other in a cycle. For example:

- Agent 1 waits for Agent 2.
- Agent 2 waits for Agent 3.
- Agent 3 waits for Agent 1.

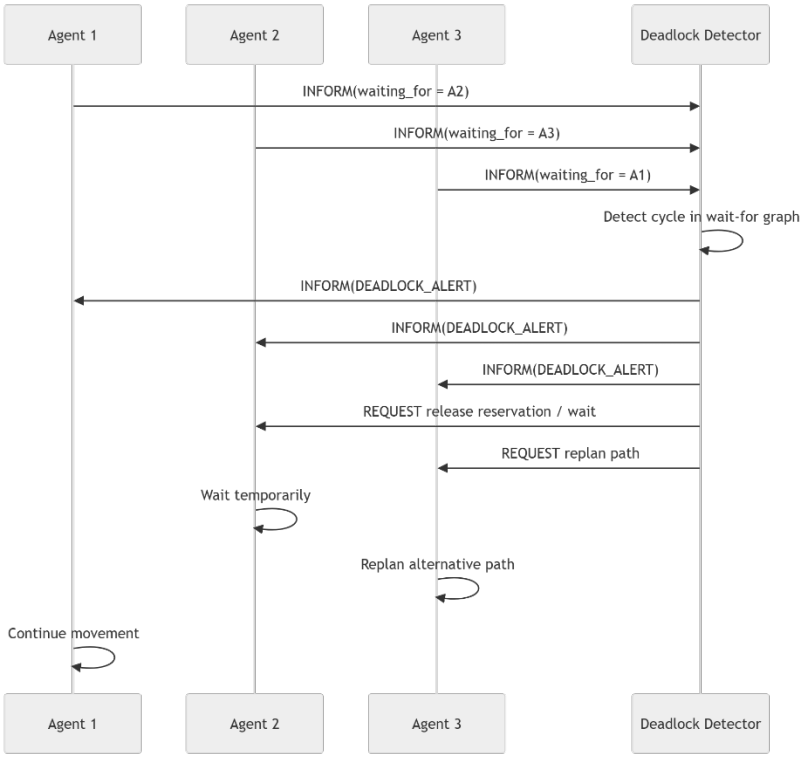
The system detects this using a wait-for graph:

- Each node represents an agent.
- An edge Agent A → Agent B means Agent A is waiting for Agent B.
- A cycle in the graph means a deadlock exists.

When a deadlock is detected:

1. Agent broadcasts DEADLOCK_ALERT.
2. Involved agents freeze movement for one step.
3. A resolving agent is selected using priority.
4. One or more lower-priority agents release reservations.
5. Selected agents replan paths.
6. Agents resume movement.

Deadlock Resolution Sequence Diagram



2.9 Communication Assumptions and Limitations

The simulation assumes:

- Communication is reliable.
- Message delay is negligible.
- Agents can broadcast to nearby agents or all agents.
- Messages are processed once per simulation time step.
- Future task arrivals are unknown.
- Agents do not have perfect global knowledge of all future events.

These assumptions are acceptable for the SDD because the project scope is simulation-based, not physical robot deployment. The SRS also states that physical deployment and hardware constraints are outside the project scope.

However, the limitation is obvious: this communication model is optimistic. In a real warehouse, network delay, packet loss, and sensor errors would matter. For this project, ignoring those issues is the right choice because adding them now would make the design unnecessarily complicated.

2.10 Summary

The interaction and communication design is based on decentralized message exchange between warehouse robot agents. The main communication mechanisms are:

- FIPA-ACL inspired messages.
- Auction-based task negotiation.
- Movement intention broadcasting.
- Reservation-table coordination.
- Priority-based conflict resolution.
- Wait-for graph deadlock communication.

3. Coordination Mechanism Logic

3.1 Coordination Overview

The coordination mechanism defines how warehouse robot agents cooperate without a centralized controller. Each agent makes local decisions, but all agents follow the same rules. This allows the system to produce organized global behavior from decentralized local actions.
The coordination logic is built from five connected mechanisms:

1. **Decentralized task allocation** using an auction mechanism.
2. **Congestion-aware bidding** to avoid overloading narrow areas.
3. **Cooperative A*** path planning using a reservation table.
4. **Priority-based conflict resolution** for movement conflicts.
5. **Wait-for graph deadlock detection** with local replanning.

This matches the project direction from the One Pager: decentralized coordination, congestion-aware task allocation, Cooperative A*, fallback path planning, wait-for graph deadlock detection, local intent communication, and dynamic replanning.

3.2 Coordination Goal

The purpose of the coordination mechanism is to reach a stable cooperative state where:

- each available task is assigned to at most one agent
- each active agent has a valid task or remains idle without blocking others
- no two agents occupy the same cell at the same time
- agents avoid edge conflicts, meaning two agents cannot swap positions in the same time step
- congestion is reduced when possible
- deadlocks are detected and resolved
- agents keep making progress toward task completion.

3.3 Global Simulation Coordination Cycle

The system operates in synchronous discrete time steps. At each time step, all agents perform perception, reasoning, coordination, and action execution.

Algorithm:

Algorithm 1: Global Simulation Coordination Cycle

Input:
Grid G
Agent set A = {a1, a2, ..., an}
Task pool T
Reservation table R
Current time step t

For each time step t:

1. Generate new tasks according to the task arrival process.

2. Update the shared task pool:
   - Add new tasks.
   - Remove completed tasks.
   - Keep assigned tasks linked to their assigned agents.

3. For each agent ai:
   - Perceive local environment.
   - Receive communication messages.
   - Update beliefs.

4. Run decentralized task allocation:
   - Idle agents evaluate available tasks.
   - Agents submit bids.
   - Winning agents assign tasks to themselves.

5. For each assigned agent ai:
   - Compute or update path using Cooperative A*.
   - Reserve planned cells in the reservation table.

6. For each moving agent ai:
   - Broadcast next movement intention.

7. Detect movement conflicts:
   - Vertex conflicts.
   - Edge conflicts.
   - Occupancy conflicts.
   - Reservation conflicts.

8. Resolve conflicts using priority rules:
   - Higher-priority agent keeps movement.
   - Lower-priority agent waits or replans.

9. Detect deadlocks:
   - Build wait-for graph.
   - Search for cycles.

10. Resolve deadlocks if detected:
   - Select resolving agent.
   - Release or modify reservations.
   - Trigger replanning.

11. Execute one action per agent:
   - Move, wait, pick up, deliver, bid, or communicate.

12. Update logs and performance metrics.

3.4 Task Allocation Algorithm

Task allocation is performed using a decentralized auction. Each idle agent evaluates available tasks and computes a bid. The agent with the lowest valid bid wins the task.

The SRS already defines this logic: a task becomes available, agents evaluate it, submit bids, the winning bid is selected using a consistent rule, and deterministic tie-breaking is applied if bids are equal.

3.4.1 Bid Cost Function

Each agent computes a bid cost for task T_k:

$$BidCost_i(T_k) = \alpha \cdot D_i(T_k) + \beta \cdot C_i(T_k) + \gamma \cdot W_i + \delta \cdot R_i(T_k)$$

Where:

Symbol	Meaning
$D_i(T_k)$	Estimated travel distance from agent position to pick up and then to drop-off.
$C_i(T_k)$	Estimated congestion along the expected route.
W_i	Current workload of the agent. Idle agents have lower workload cost.
$R_i(T_k)$	Estimated reservation delay caused by other agents planned paths.
$\alpha, \beta, \gamma, \delta$	Tunable weights control the importance of each cost component.

The lowest bid represents the agent expected to complete the task with the lowest cost.

3.4.2 Task Allocation Pseudocode

Algorithm 2: Decentralized Auction-Based Task Allocation

Input:
Available task set T_unassigned
Agent set A
Current time step t

For each unassigned task Tk in T_unassigned:

1. Broadcast task announcement:
   CFP(Tk)

2. For each idle agent ai:
   Estimate distance cost Di(Tk)
   Estimate congestion cost Ci(Tk)
   Estimate workload cost Wi
   Estimate reservation delay Ri(Tk)

   Compute:
   BidCost_i(Tk) =
   α * Di(Tk)
   + β * Ci(Tk)
   + γ * Wi
   + δ * Ri(Tk)

   Send PROPOSE(Tk, BidCost_i)

3. Collect all valid bids.

4. Select winner:
   winner = agent with minimum BidCost

5. If two or more bids are equal:
   Choose the agent with the lowest agent_id.

6. Assign task Tk to winner.

7. Update task state:
   unassigned → assigned

8. Send:
   ACCEPT_PROPOSAL to winner
   REJECT_PROPOSAL to losing agents

This gives us a practical cooperation mechanism. It is not just “agents communicate”; it says exactly how the task owner is selected.

3.5 Cooperative A* Path Planning Logic

After an agent wins a task, it plans a path to the pickup location and then to the drop-off location. The primary planner is **Cooperative A***.

Normal A* searches only spatial cells. Cooperative A* searches over **cell + time step**, which means a state is represented as:

$$State = (x, y, t)$$

This allows agents to avoid not only current obstacles, but also future conflicts with cells reserved by other agents.

3.5.1 Reservation Table

The reservation table stores future planned occupation of cells.

Each entry is:

$$\begin{aligned}&Reservation = \{\\&\quad agent\_id,\\&\quad cell = (x, y),\\&\quad time\_step,\\&\quad previous\_cell,\\&\quad reservation\_type\\&\}\end{aligned}$$

Two reservation types are used:

Type	Meaning
Vertex reservation	A cell is reserved by an agent at a specific time step.
Edge reservation	A movement from one cell to another is reserved to prevent position swapping.

3.5.2 Cooperative A* Pseudocode

Algorithm 3: Cooperative A* Path Planning

```
Input:
   Start cell s
   Goal cell g
   Current time t0
   Grid G
   Reservation table R

Output:
   Conflict-free path P

1. Initialize open set with node:
      (s.x, s.y, t0)

2. Initialize closed set as empty.

3. While open set is not empty:

      current = node with lowest f value

      If current.cell == g:
         Return reconstructed path P

      Add current to closed set.

      For each possible action:
         - move up
         - move down
         - move left
         - move right
         - wait

         next_cell = result of action
         next_time = current.time + 1

         If next_cell is outside grid:
            reject successor

         If next_cell is obstacle:
            reject successor

         If next_cell is reserved at next_time:
            reject successor

         If edge movement is reserved in opposite direction:
            reject successor
```

Compute:
g_cost = path cost so far
h_cost = ManhattanDistance(next_cell, goal)
congestion_penalty = local congestion around next_cell
f_cost = g_cost + h_cost + congestion_penalty

Add successor to open set.

4. If no path is found:
Return failure and trigger fallback planning.

The important part: the planner includes **wait** as a valid action. Without waiting, some narrow-corridor situations become impossible to solve even when they are solvable.

3.6 Movement Conflict Detection

After agents plan their paths, they broadcast short-term movement intentions. Each agent checks whether its next action conflicts with another agent's intention.
The system detects three main conflict types:

Conflict Type	Condition	Example
Vertex conflict	Two agents intend to enter the same cell at the same time.	A1 and A2 both move to (4,5) at t+1.
Edge conflict	Two agents attempt to swap positions.	A1 moves (4,5) → (4,6) while A2 moves (4,6) → (4,5).
Blocking conflict	Agent wants to enter a cell occupied by another agent that is not moving.	A1 wants to enter A2's cell, but A2 waits.

Conflict Detection Pseudocode

Algorithm 4: Movement Conflict Detection

```
Input:
   Movement intentions I
   Current agent positions P

For each pair of agents (ai, aj):

   1. Check vertex conflict:
      If ai.next_cell == aj.next_cell:
         conflict = VERTEX_CONFLICT

   2. Check edge conflict:
      If ai.current_cell == aj.next_cell
      AND ai.next_cell == aj.current_cell:
         conflict = EDGE_CONFLICT

   3. Check blocking conflict:
      If ai.next_cell == aj.current_cell
      AND aj.action == WAIT:
         conflict = BLOCKING_CONFLICT

   4. If conflict exists:
      Add conflict to conflict list.
```

3.7 Priority-Based Conflict Resolution

When a conflict is detected, the system uses deterministic local priority rules. The goal is to avoid random behavior and make simulation results reproducible.

Priority score:

$$Priority_i = \lambda_1 \cdot CarryingItem_i + \lambda_2 \cdot WaitingTime_i + \lambda_3 \cdot Progress_i - \lambda_4 \cdot AgentID_i$$

Where:

Term	Meaning
$CarryingItem_i$	Higher priority if the agent already picked up an item.
$WaitingTime_i$	Higher priority if the agent has waited longer.
$Progress_i$	Higher priority if the agent is close to finishing the task.
$AgentID_i$	Used only as deterministic tiebreaker.

The conflict resolution rules are:

1. Agent carrying an item has priority over an agent going to pickup.
2. Agent that waited longer has priority.
3. Agent closer to task completion has priority.
4. If still tied, lower agent_id wins.
5. Losing agent waits for one step.
6. If the same agent waits more than WAIT_THRESHOLD, it triggers replanning.

Conflict Resolution Pseudocode

Algorithm 5: Priority-Based Conflict Resolution

Input:
Conflict list C
Agents A

For each conflict c in C:

1. Identify involved agents.

2. Compute priority score for each involved agent.

3. Select highest-priority agent as winner.

4. Winner keeps its planned movement.

5. All losing agents:
   - cancel movement for current time step
   - execute WAIT
   - increment waiting_time

6. If waiting_time > WAIT_THRESHOLD:
   - trigger replanning

3.8 Congestion-Aware Coordination

Congestion is handled at two levels:

1. **Before task assignment** — congestion affects the bid cost.
2. **During movement** — congestion affects path planning and replanning.

A congestion score is calculated for each cell or region:

$$Congestion(c,t) = \frac{AgentsNear(c,t)}{Capacity(c)}$$

Where:

- $AgentsNear(c,t)$ is the number of agents in or near cell $c$at time t.
- $Capacity(c)$ is the maximum desirable number of agents near that cell or region.

If congestion exceeds a threshold:

```
If Congestion(c,t) > CONGESTION_THRESHOLD:
  increase path cost through c
  avoid assigning new tasks through c when possible
  trigger replanning if agent is repeatedly delayed
```

3.9 Dynamic Replanning Logic

Agents replan when the original path is no longer efficient or safe.
Replanning is triggered when:

- the next cell is blocked,
- a reservation is rejected,
- congestion exceeds the threshold,
- the agent waits more than WAIT_THRESHOLD,
- a deadlock is detected,
- the agent receives a conflict warning,
- the path becomes invalid.

Replanning Pseudocode

```
Algorithm 6: Dynamic Replanning
Input:
Agent ai
Current position pos_i
Current goal goal_i
Reservation table R
1. Release future reservations owned by ai.
2. Recompute path using Cooperative A*:
new_path = CooperativeAStar(pos_i, goal_i, R)
3. If new_path exists:
Reserve new path.
Replace old path with new_path.
Reset waiting counter.
Continue execution.
4. Else:
Use fallback planner:
Independent A* + priority-based conflict resolution.

5. If fallback also fails:
Agent waits and retries at next time step.
```

3.10 Deadlock Detection Using Wait-For Graph

A deadlock occurs when agents wait for each other in a cycle.
Example:
A1 waits for A2
A2 waits for A3
A3 waits for A1
This creates a cycle:
A1 → A2 → A3 → A1
The wait-for graph is defined as:

$$WFG = (V, E)$$

Where:

- V is the set of agents.
- E is the direct waiting edges.
- (a_i, a_j) means that agents a_i is waiting for agent a_j.

Deadlock Detection Pseudocode
Algorithm 7: Deadlock Detection

Input:
Agents A
Waiting relationships W

1. Create empty directed graph WFG.

2. For each agent ai:
If ai is waiting for aj:
Add edge ai → aj

3. Run cycle detection on WFG.

4. If a cycle exists:
deadlock_detected = true
involved_agents = agents in cycle

5. Else:
deadlock_detected = false

3.11 Deadlock Resolution Logic

When a deadlock is detected, the system breaks the cycle by forcing one or more agents to release reservations, wait, or replan.

Resolution rules:

1. Identify all agents in the deadlock cycle.
2. Select the agent with the highest priority to continue.
3. Lower-priority agents release future reservations.
4. One lower-priority agent may move aside if an alternative cell exists.
5. Agents replan paths using updated reservations.
6. If no alternative exists, agents wait in deterministic order.

Deadlock Resolution Pseudocode

Algorithm 8: Deadlock Resolution

Input:
Deadlock cycle D = {a1, a2, ..., ak}
Reservation table R

1. Freeze involved agents for one simulation step.

2. Compute priority for each agent in D.

3. Select highest-priority agent a_high.

4. For each agent ai in D where ai != a_high:
Release future reservations of ai.
Set ai.state = REPLANNING

5. Allow a_high to keep its reservation if valid.

6. For each replanning agent:
Try to find an alternative path using Cooperative A*.

7. If alternative path exists:
Reserve new path.
Resume movement.

8. If no path exists:
Agent waits according to deterministic order.

9. Continue until no cycle remains in the wait-for graph.

3.12 Full Agent-Level Coordination Algorithm

This is the algorithm each agent follows internally.

Algorithm 9: Agent Coordination Step

Input:
Agent ai
Current grid state G
Task pool T
Reservation table R
Received messages M
Current time step t

1. Update beliefs:
   Read current position.
   Read task state.
   Read nearby agents.
   Read received messages.
   Read reservation table.

2. If ai has no assigned task:
   If unassigned tasks exist:
      Participate in auction.
   Else:
      Remain idle without blocking active paths.

3. If ai receives a task assignment:
   Set current goal to pickup location.
   Plan path using Cooperative A*.
   Reserve path cells.

4. If ai reaches pickup location:
   Execute PICKUP.
   Set current goal to drop-off location.
   Plan path to drop-off.
   Reserve path cells.

5. If ai reaches drop-off location:
   Execute DELIVER.
   Mark task as completed.
   Release remaining reservations.
   Become IDLE.

6. If ai is moving:
   Select next cell from path.
   Broadcast movement intention.

7. Check received movement intentions:
   If conflict is detected:
      Apply priority-based conflict resolution.

8. If ai loses conflict:
   Execute WAIT.
   Increase waiting_time.

9. If waiting_time exceeds threshold:
   Trigger replanning.

10. If deadlock alert is received:
   Participate in deadlock resolution.

11. Execute selected action.

12. Update logs:
    action taken
    task state
    waiting time
    conflict status
    path status

3.13 Why This Coordination Mechanism Is Suitable

This coordination mechanism is suitable because it directly addresses the real problems in the project:

Problem	Mechanism Used
Multiple agents want the same task	Auction-based allocation
Some routes are overloaded	Congestion-aware bid and path cost
Agents may collide	Movement intention and reservation table
Agents may swap positions	Edge reservation
Narrow corridors can create blocking	Priority rules and waiting
Agents may get stuck in cycles	Wait-for graph deadlock detection
Dynamic tasks and congestion appear	Dynamic replanning

4. System Implementation Design

4.1 Implementation Design Overview

The implementation design describes how the warehouse MAS will be translated into software components. The system will be implemented as an **agent-based simulation**, where each warehouse robot is represented as an autonomous software agent operating inside a shared grid environment.

The implementation must support:

1. Dynamic task generation.
2. Decentralized task allocation.
3. Agent-level path planning.
4. Reservation-based movement coordination.
5. Conflict detection and resolution.
6. Deadlock detection.
7. Metrics collection.
8. Real-time visualization.

This section is important because the SDD assignment explicitly requires **UML class diagrams** and a description of the **technical stack used for implementation**.

4.2 Selected Technical Stack

The selected implementation stack below is the reference design for this project. It is intentionally specific so the SDD describes one concrete build path rather than a menu of alternatives.

Layer	Selected Technology	Purpose
Programming language	Python 3.x	Main implementation language for the simulator, agent logic, analytics export, and experiment control.
Simulation model	Custom synchronous grid-based simulator in Python	Provides exact control over discrete time steps, shared reservations, conflict resolution, and reproducible scenario execution.
Path planning	Custom A* and Cooperative A* implementation	Computes valid spatial and time-space paths while enforcing reservation and conflict constraints.
Data handling	Python dataclasses, enums, dictionaries, and lists	Stores agents, tasks, reservations, messages, logs, and metrics with simple deterministic in-memory structures.
Visualization	Pygame-based real-time dashboard	Renders the warehouse grid, agents, tasks, planned paths, congestion overlays, controls, and live validation state.
Analytics	Structured CSV/JSON logs plus Python analysis scripts	Records experiment traces and supports scenario comparison, metric aggregation, and post-run validation.
UML design	draw.io UML class diagrams exported as PNG	Documents class structure and relationships using a standard engineering notation that can be embedded in the report.

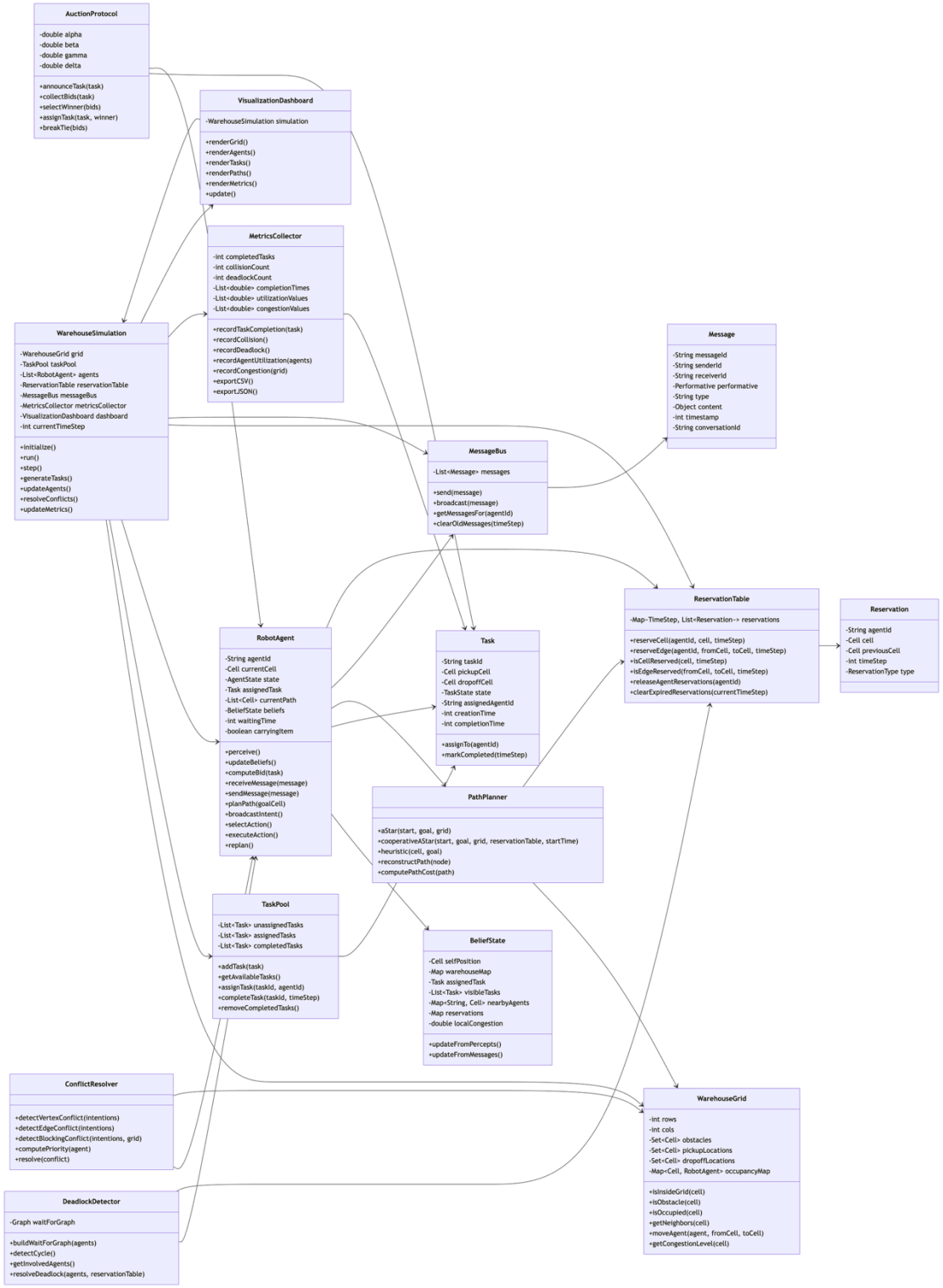
This stack is chosen on purpose: a custom synchronous simulator is better than a generic ABM framework for reservation-based Cooperative A*, deterministic tie-breaking, and step-by-step debugging, while Pygame keeps the dashboard practical for real-time observation and control.

4.3 Main Software Components

The implementation is divided into the following main components:

Component	Responsibility
WarehouseSimulation	Controls the main simulation loop and coordinates global time steps.
WarehouseGrid	Represents the grid, obstacles, pickup/drop-off cells, and cell occupancy.
RobotAgent	Represents an autonomous warehouse robot agent.
Task	Represents a pickup-and-drop-off task.
TaskPool	Stores and manages unassigned, assigned, and completed tasks.
AuctionProtocol	Handles decentralized task bidding and task winner selection.
PathPlanner	Computes paths using A* and Cooperative A*.
ReservationTable	Stores future cell and edge reservations for collision-free planning.
ConflictResolver	Detects and resolves vertex, edge, and blocking conflicts.
DeadlockDetector	Builds a wait-for graph and detects cyclic waiting.
MessageBus	Supports communication between agents.
MetricsCollector	Records throughput, completion time, congestion, utilization, and deadlocks.
VisualizationDashboard	Renders the simulation state to the observer.

4.4 UML Class Diagram
Link to a clearer diagram



4.5 Class Descriptions

4.5.1 WarehouseSimulation

WarehouseSimulation is the main controller of the simulation. It does not act as a central decision-maker for the agents. Its role is only to advance the simulation time step, update the environment, call agent decision cycles, resolve global consistency issues, and collect metrics.

Responsibilities:

- Initialize the grid, agents, task pool, reservation table, and metrics collector.
- Run the simulation loop.
- Generate new tasks dynamically.
- Trigger each agent's decision cycle.
- Execute actions synchronously.
- Update visualization and analytics. .

4.5.2 WarehouseGrid

WarehouseGrid represents the physical/logical warehouse environment.

Responsibilities:

- Store grid dimensions.
- Store obstacle cells.
- Store pickup and drop-off locations.
- Track current agent occupancy.
- Validate movement.
- Return neighboring cells.
- Compute local congestion.

4.5.3 RobotAgent

RobotAgent is the core MAS entity. Each robot agent is autonomous and has its own beliefs, state, assigned task, current path, waiting time, and communication behavior.

Responsibilities:

- Perceive the environment.
- Update internal beliefs.
- Participate in task auctions.
- Compute bids.
- Plan paths.
- Broadcast movement intentions.
- Detect local conflicts.
- Execute movement, waiting, pickup, and delivery actions.
- Replan when blocked or delayed.

4.5.4 BeliefState

BeliefState stores the internal knowledge used by each agent.

It includes:

- Current self-position.
- Known warehouse map.
- Assigned task.
- Visible or available tasks.
- Nearby agents.
- Received reservations.
- Local congestion estimate.

4.5.5 Task

Task represents a warehouse job.
Each task contains:

- Task ID.
- Pickup cell.
- Drop-off cell.
- Current state.
- Assigned agent ID.
- Creation time.
- Completion time.

Task states:
UNASSIGNED → ASSIGNED → COMPLETED

4.5.6 TaskPool

TaskPool manages all tasks in the system.
Responsibilities:

- Add newly generated tasks.
- Return available unassigned tasks.
- Assign tasks to winning agents.
- Mark tasks as completed.
- Remove completed tasks from the active pool.

4.5.7 AuctionProtocol

AuctionProtocol handles task negotiation.
Responsibilities:

- Announce available tasks.
- Collect bids from agents.
- Compare bid costs.
- Select the winning agent.
- Apply deterministic tie-breaking.
- Update task assignment.

Even though this class exists in the software design, it does not necessarily mean the system is centralized. It can be implemented as a protocol object used by agents or as a simulation-level helper that models decentralized message exchange.
The key rule: **the auction selects the lowest valid bid**, where the bid considers distance, congestion, workload, and reservation delay.

4.5.8 PathPlanner

PathPlanner implements movement planning.
Responsibilities:

- Run standard A*.
- Run Cooperative A*.
- Use Manhattan distance as the heuristic.
- Check grid constraints.
- Check reservation constraints.
- Return a valid path or failure.

The planner should support both:

1. **Primary planner** — Cooperative A* with reservations.
2. **Fallback planner** — independent A* with priority-based conflict resolution.

4.5.9 ReservationTable

ReservationTable stores future planned cell and edge reservations.
Responsibilities:

- Reserve a cell for an agent at a future time step.
- Reserve an edge movement to prevent position swapping.
- Check whether a cell is already reserved.
- Check whether an edge conflict exists.
- Release reservations when agents replan or complete tasks.
- Remove expired reservations.

This component is essential for Cooperative A*. Without it, the planner cannot prevent future collisions.

4.5.10 ConflictResolver

ConflictResolver handles movement conflicts.
Responsibilities:

- Detect vertex conflicts.
- Detect edge conflicts.
- Detect blocking conflicts.
- Compute agent priority.
- Decide which agent moves and which agent waits or replans.

4.5.11 DeadlockDetector

DeadlockDetector identifies cyclic waiting situations.
Responsibilities:

- Build the wait-for graph.
- Add an edge when one agent is waiting for another.
- Detect cycles.
- Return the agents involved in the deadlock.
- Trigger deadlock resolution.

4.5.12 MessageBus

MessageBus simulates communication between agents.
Responsibilities:

- Send direct messages.
- Broadcast messages.
- Store messages for one simulation step.
- Deliver messages to target agents.
- Clear outdated messages.

This component supports FIPA-ACL-style communication from Section 2.

4.5.13 MetricsCollector

MetricsCollector records simulation data for evaluation.
Responsibilities:

- Track completed tasks.
- Track average task completion time.
- Track collision count.
- Track deadlock count and resolution time.
- Track agent utilization.
- Track congestion.
- Export logs to CSV or JSON.

4.5.14 VisualizationDashboard

VisualizationDashboard renders the simulation to the observer.
Responsibilities:

- Draw the warehouse grid.
- Show agents.
- Show obstacles.
- Show pickup and drop-off points.
- Show assigned tasks.
- Show planned paths.
- Show congestion areas.
- Show live metrics.

The dashboard is not just decoration. It is needed for validation because it helps verify whether agents are avoiding collisions, resolving congestion, and completing tasks correctly.

4.6 Main Enumerations

The design should also include simple enumerations for system state consistency.

- AgentState:
  - IDLE
  - BIDDING
  - ASSIGNED
  - MOVING_TO_PICKUP
  - PICKING_UP
  - MOVING_TO_DROPOFF
  - DELIVERING
  - WAITING
  - REPLANNING
  - DEADLOCK_RECOVERY
- TaskState:
  - UNASSIGNED
  - ASSIGNED
  - COMPLETED
- Performative:
  - INFORM
  - REQUEST

- CFP
- PROPOSE
- ACCEPT_PROPOSAL
- REJECT_PROPOSAL

❖ ReservationType:
- VERTEX
- EDGE

❖ ConflictType:
- VERTEX_CONFLICT
- EDGE_CONFLICT
- BLOCKING_CONFLICT
- RESERVATION_CONFLICT

These enums make the implementation cleaner and reduce bugs caused by inconsistent string values.

4.7 Main Simulation Flow

The software execution flow is:

1. Initialize WarehouseSimulation.

2. Load or create WarehouseGrid.

3. Create RobotAgent objects.

4. Create TaskPool, ReservationTable, MessageBus, MetricsCollector, and VisualizationDashboard.

5. For each simulation time step:

   a. Generate new tasks.

   b. Add tasks to TaskPool.

   c. Deliver communication messages through MessageBus.

   d. Each RobotAgent updates beliefs.

   e. Idle agents participate in AuctionProtocol.

   f. Assigned agents compute paths using PathPlanner.

   g. Agents reserve cells using ReservationTable.

   h. Agents broadcast movement intentions.

   i. ConflictResolver detects and resolves conflicts.

   j. DeadlockDetector checks for cyclic waiting.

k. Agents execute one action.

l. WarehouseGrid updates positions.

m. TaskPool updates task states.

n. MetricsCollector records system state.

o. VisualizationDashboard renders the new state.

6. Stop simulation when max time is reached or validation scenario ends.

7. Export metrics and logs.

4.8 SRS-to-SDD Traceability Snapshot

The table below makes the SRS-to-SDD alignment explicit. It shows how the design choices in this document realize the most important requirements from the SRS, which makes grading and later implementation traceability much easier.

SRS requirement / concern	SDD realization
FR-3, FR-11, FR-12, FR-13 — consistent task assignment and single-task execution	Handled by AuctionProtocol, TaskPool, deterministic tie-breaking, the RobotAgent state machine, and the task lifecycle design (Sections 1.4, 1.5, 2.4, 3.4, 4.5.3, 4.5.6, 4.5.7).
FR-5 and FR-6 — collision avoidance and valid movement	Handled by WarehouseGrid constraint checking, ReservationTable, Cooperative A*, and ConflictResolver so agents cannot legally move into obstacles, occupied cells, or conflicting edge swaps (Sections 3.5, 3.6, 3.7, 4.5.2, 4.5.8, 4.5.9, 4.5.10).
FR-7 — deadlock detection and recovery	Handled by the wait-for-graph design, deadlock communication protocol, DeadlockDetector component, and deterministic recovery algorithm (Sections 2.8, 3.10, 3.11, 3.12, 4.5.11).
FR-8 and FR-17 — congestion-aware behavior	Handled by congestion-aware bid cost, local congestion beliefs, replanning triggers, congestion visualization, and congestion analytics (Sections 1.3.2, 3.4.1, 3.8, 5.10, 6.7.7).
FR-14 and FR-15 — path planning and dynamic replanning	Handled by the primary Cooperative A* planner, fallback A*, reservation release rules, and replanning logic when routes are blocked or overloaded (Sections 1.3.3, 1.3.4, 3.5, 3.9, 4.5.8, 4.5.9).
FR-19 and FR-20 — communication usage and task completion signaling	Handled by FIPA-ACL-style messages, movement-intent exchange, MessageBus delivery, and structured completion/event logs (Sections 2.2, 2.3, 2.5, 4.5.12, 6.2, 6.4).
SRS Section 5 — simulator validation and observability	Handled by the dashboard layout, event log, observer validation table, scenario controls, metrics collector, and scenario-level analytics (Sections 5.5 through 5.16 and Section 6.9).→ assigned → completed.

4.9 Design Notes and Constraints

Design intentionally avoids unnecessary production-level complexity.
Not included:

- Real robot hardware.
- Real network communication.
- Cloud services.
- Database server.
- Reinforcement learning in the core design.
- Physical warehouse integration.

4.10 Summary

The implementation design is based on a modular Python agent-based simulation. The most important classes are:

- RobotAgent
- WarehouseSimulation
- WarehouseGrid
- TaskPool
- AuctionProtocol
- PathPlanner
- ReservationTable
- ConflictResolver
- DeadlockDetector
- MessageBus
- MetricsCollector
- VisualizationDashboard

5. Simulator & Visualization Architecture

5.1 Overview

The simulator and visualization architecture defines how the multi-agent warehouse system will be executed, observed, inspected, and validated.
The simulator is designed as a **discrete-time agent-based simulation engine**. Each robot is represented as an autonomous agent, and the warehouse is represented as a 2D grid environment. At every simulation time step, agents perceive the environment, communicate, make decisions, reserve paths, resolve conflicts, and execute one action. This design follows the SRS requirement that the simulator must model the warehouse environment, execute agent behaviors in real time using discrete time steps, support multiple concurrent agents, and use a configurable random seed for reproducible experiments.
The design also matches the MAS lecture definition of Agent-Based Modeling, where a system is represented as autonomous interacting agents whose actions are simulated to observe system-level effects.

5.2 Simulation Engine Architecture

The simulation engine is responsible for advancing the system from one time step to the next. It does not make decisions for the agents. It only provides the environment, applies rules, executes actions, and records outcomes.
The simulation engine contains the following modules:

Module	Responsibility
SimulationController	Runs the simulation loop and manages start, pause, reset, and step-by-step execution.
WarehouseGrid	Stores the grid layout, obstacles, pickup cells, drop-off cells, and current occupancy.
AgentManager	Maintains the list of active robot agents and calls their decision cycle.
TaskGenerator	Creates new tasks according to the selected task generation rate.
TaskPool	Stores unassigned, assigned, and completed tasks.
MessageBus	Delivers agent communication messages during each time step.
ReservationTable	Stores future cell and edge reservations.
ConflictMonitor	Detects invalid movement attempts, collision risks, and blocked agents.
DeadlockMonitor	Detects cyclic waiting using a wait-for graph.
MetricsCollector	Records metrics and events for validation.
VisualizationDashboard	Renders the current simulation state to the observer.

The important point: **the engine is not a centralized controller**. It is a simulation platform. The agents still decide their own tasks, paths, and actions.

5.3 Discrete-Time Simulation Loop

The simulator runs in synchronous discrete time steps:
Algorithm 10: Simulation Engine Loop

Input:
Grid layout G
Agent set A
Task pool T
Reservation table R
Simulation parameters P
Random seed S

1. Initialize simulation using random seed S.

2. Create warehouse grid:
- free cells
- obstacles
- pickup locations
- drop-off locations

3. Spawn agents at valid starting cells.

4. Initialize task pool, message bus, metrics collector, and dashboard.

5. For each time step t = 1 to T_max:

a. If simulation is paused:
wait until resumed.

b. Generate new tasks according to task arrival rate.

c. Add new tasks to the task pool.

d. Deliver messages from previous step to agents.

e. For each agent ai:
ai.perceive()
ai.updateBeliefs()
ai.selectGoal()
ai.planOrReplanIfNeeded()
ai.broadcastIntent()

f. Collect movement intentions.

g. Check reservation table and conflict conditions.

h. Resolve movement conflicts.

i. Detect and resolve deadlocks.

j. Execute one approved action for each agent.

k. Update grid occupancy.

l. Update task states:
unassigned → assigned → completed

m. Clear expired reservations and old messages.

n. Record logs and metrics.

o. Render dashboard.

6. Stop when:
- maximum time step is reached, or
- validation scenario ends, or
- user stops the simulation.

7. Export logs and metric files.

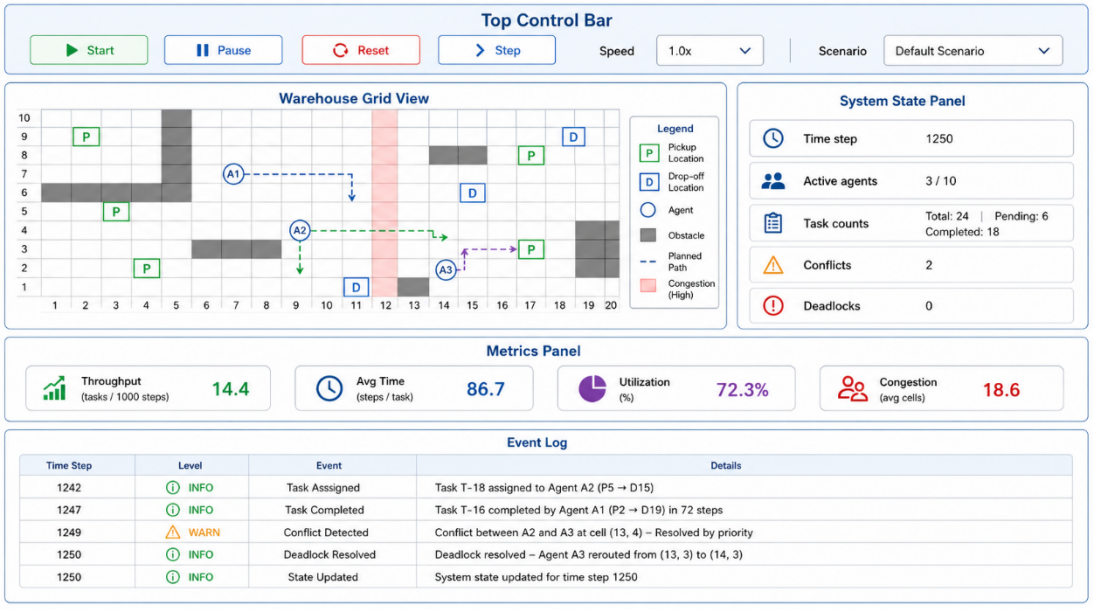
5.4 Simulation Parameters

The simulator should expose configurable parameters so different validation scenarios can be tested.

Parameter	Description
grid_rows	Number of grid rows.
grid_cols	Number of grid columns.
num_agents	Number of robot agents in the warehouse.
num_obstacles	Number or layout of blocked cells.
pickup_locations	Predefined pickup points.
dropoff_locations	Predefined delivery points.
task_generation_rate	Probability or frequency of new task creation.
max_time_steps	Maximum simulation length.
random_seed	Seed used for reproducibility.
agent_speed	Number of cells an agent may move per time step. Usually 1.
congestion_threshold	Threshold for marking a region as congested.
wait_threshold	Number of waiting steps before replanning/deadlock handling.
visualization_speed	Delay between rendered time steps.

5.5 Dashboard Layout

The visualization dashboard should be divided into four main areas:



This layout is practical because the observer can see both the **spatial behavior** and the **numeric/system state** at the same time.

5.6 Grid Rendering Design

The main visualization is the warehouse grid. Each cell has a visual meaning.

Grid Element	Rendered As
Free cell	Empty/light cell.
Obstacle	Dark blocked cell.
Pickup location	Cell marked with P.
Drop-off location	Cell marked with D.
Robot agent	Colored circle or square with agent ID.
Agent carrying item	Agent icon with border or package symbol.
Planned path	Thin line or semi-transparent trail.
Reserved future cell	Light highlighted cell.
Congested area	Heatmap intensity or highlighted region.
Conflict warning	Red outline around conflict cell.
Deadlock area	Flashing or emphasized group of cells/agents.

5.7 Agent Rendering

Each agent should be rendered with enough information to understand its current behavior.

For each robot agent, the dashboard should show:

Visual Feature	Meaning
Agent ID label	Identifies the robot.
Unique color	Distinguishes one agent from another.
Current cell	Shows physical position.
Direction arrow	Shows next intended movement.
Path line	Shows planned route.
Status marker	Shows idle, moving, waiting, replanning, or deadlock recovery.
Package icon/border	Shows whether the agent is carrying an item.

Example agent labels:

- A1: IDLE
- A2: MOVING_TO_PICKUP
- A3: WAITING
- A4: DEADLOCK_RECOVERY

This is necessary because a viewer should not need to guess what the agents are doing. If the visualization only shows moving dots, it is weak.

5.8 Task Visualization

Tasks should be visible both on the grid and in the side panel.

Each task should display:

Task State	Visual Representation
UNASSIGNED	Pickup/drop-off pair shown with neutral color or dashed line.
ASSIGNED	Task shown using the assigned agent's color.
COMPLETED	Removed from active grid view and counted in metrics.

The task lifecycle should be shown clearly:

UNASSIGNED → ASSIGNED → COMPLETED

5.9 Path and Reservation Visualization

The dashboard should show both the planned path and the reserved future cells.
Path visualization:

- A line from the agent's current position to its target.
- Different style for path to pickup and path to drop-off.
- Updated when the agent replans.

Reservation visualization:

- Future reserved cells highlighted lightly.
- Optional labels showing reservation time step.
- Edge reservations are shown as arrows between cells.

This is important because the project uses Cooperative A*. If the reservation table is invisible, the observer cannot verify whether Cooperative A* is actually being used or whether agents are just moving randomly.

5.10 Congestion Visualization

Congestion should be displayed as a heatmap overlay on the grid.
Congestion value can be calculated as:

$$Congestion(cell, t) = \frac{number\ of\ agents\ within\ local\ neighborhood\ of\ cell}{local\ capacity\ threshold}$$

Rendering logic:

Congestion Level	Visualization
Low	No highlight.
Medium	Light highlight.
High	Strong highlight / warning outline.

The dashboard should also show a global CongestionIndex in the metrics panel.

5.11 Conflict and Deadlock Visualization

The visualization should explicitly show conflicts and deadlocks.
Conflict Rendering
When a conflict is detected:

- Mark the conflict cell.
- Show involved agents.
- Display conflict type:
  - vertex conflict,
  - edge conflict,
  - blocking conflict,
  - reservation conflict.
- Log the selected resolution:
  - priority winner,
  - waiting agent,
  - replanning agent.

Example event log:
[t=42] VERTEX_CONFLICT: A2 and A5 requested cell (6,8). Winner: A2. A5 waits.

Deadlock Rendering
When a deadlock is detected:

- Highlight involved agents.
- Show wait-for relation arrows if possible.
- Display DEADLOCK_ALERT in the event log.
- Show when resolution completes.

Example event log:
[t=77] DEADLOCK_DETECTED: A1 -> A3 -> A4 -> A1.
[t=78] DEADLOCK_RESOLUTION: A4 replans, A1 keeps priority.
[t=84] DEADLOCK_RESOLVED: all agents resumed movement.

5.12 State and Data Panel

The side panel should display real-time state information.

Field	Description
Current time step	Current simulation tick.
Active agents	Number of agents are currently in the simulation.
Pending tasks	Number of unassigned tasks.
Assigned tasks	Number of tasks currently assigned.
Completed tasks	Number of completed deliveries.
Agent states	Count of idle, moving, waiting, replanning agents.
Current conflicts	Number and type of current conflicts.
Current deadlocks	Whether a deadlock exists.
Current throughput	Completed tasks per time unit.
Average completion time	Mean task completion duration.
Utilization	Percentage of agents actively working.
Congestion index	Average congestion level.

5.13 User Interaction Controls

The dashboard should provide the following controls:

Control	Purpose
Start	Begins or resumes the simulation.
Pause	Freezes the current simulation state.
Reset	Restarts the simulation with the selected configuration.
Step	Advances exactly one time step for detailed inspection.
Speed Slider	Controls visualization speed.
Scenario Selector	Selects baseline, high load, high density, deadlock, dynamic, or scalability scenario.
Agent Count Input	Changes the number of agents.
Task Rate Input	Changes the task generation rate.
Seed Input	Sets random seed for reproducible experiments.
Show Paths Toggle	Enables/disables path rendering.
Show Heatmap Toggle	Enables/disables congestion heatmap.

These controls are not optional decoration. They are needed for debugging and validation. Without step-by-step execution, it becomes much harder to prove why a conflict or deadlock happened.

5.14 Event Logging in Visualization

The dashboard should include a live event log.

Events to show:

- TASK_CREATED
- TASK_ASSIGNED
- TASK_PICKED_UP
- TASK_COMPLETED
- PATH_PLANNED
- PATH_REPLANNED
- MOVEMENT_INTENT
- RESERVATION_ACCEPTED
- RESERVATION_REJECTED
- CONFLICT_DETECTED
- CONFLICT_RESOLVED
- DEADLOCK_DETECTED
- DEADLOCK_RESOLVED
- INVALID_MOVE_PREVENTED

Each log entry should include:

- time_step
- event_type
- agent_id
- task_id, if relevant
- cell/location, if relevant
- short description

5.15 Observer Validation

The visualization must allow an observer to verify that the system works correctly.

Requirement	How Observer Verifies It
No two agents occupy same cell	Grid view shows unique occupancy per cell.
No edge swapping	Movement arrows and conflict logs show blocked swaps.
Valid path planning	Planned paths avoid obstacles and reserved cells.
Dynamic replanning	Path changes after blockage or congestion.
Congestion response	Heatmap shows congestion and agents route around it.
Deadlock resolution	Deadlock log shows detection and recovery.
Task lifecycle	Task panel shows unassigned → assigned → completed.
System continuity	Simulation continues while tasks are available.
Scalability	Metrics remain stable as agent count increases.

5.16 Visualization Update Cycle
The dashboard should update once per simulation time step.
Algorithm 11: Visualization Update Cycle

Input:
Current simulation state S(t)

1. Clear previous frame.

2. Draw warehouse grid.

3. Draw static elements:
- obstacles
- pickup locations
- drop-off locations

4. Draw dynamic elements:
- agents
- assigned tasks
- planned paths
- reserved cells
- congestion heatmap

5. Draw warnings:
- conflicts
- invalid movement attempts
- deadlock cycles

6. Update side panel:
- time step
- task counts
- agent states
- live metrics

7. Append new event log entries.

8. Render frame to observer.
This keeps the visualization synchronized with the simulation state and makes debugging much easier.

6. Data & Analytics Design

6.1 Overview

The data and analytics layer is responsible for recording simulation events, computing performance metrics, and validating whether the selected agent strategies work.
This section is not just for "nice graphs." It is how the system proves that the coordination mechanisms are successful. The SDD assignment explicitly requires a design of logs and metrics collection for validating agent strategies.
The analytics design must support validation of:

- task allocation efficiency,
- collision avoidance,
- congestion handling,
- path planning quality,
- deadlock detection and recovery,
- agent utilization,
- scalability under increasing agent density,
- system stability under dynamic task arrivals.

6.2 Logging Architecture

The simulator will use a structured logging system. Each important event during the simulation is stored as a log record.
The logging system has three levels:

Log Level	Purpose
Event Log	Records high-level simulation events such as task creation, assignment, completion, conflicts, and deadlocks.
Agent Log	Records each agent's decisions, state changes, bids, movements, waits, and replanning actions.
Metric Log	Records numerical metrics at each time step or at the end of each scenario.

This separation is important. If everything is dumped into one log file, the data becomes messy and hard to analyze.

6.3 Log Record Format

Each log entry should use a structured format such as CSV or JSON.

6.3.1 General Event Log Schema

⟨/⟩ JSON

```
{
 "time_step": 1250,
 "event_type": "CONFLICT_DETECTED",
 "agent_id": "A2",
 "related_agents": ["A3"],
 "task_id": "T18",
 "cell": [13, 4],
 "details": "Vertex conflict detected between A2 and A3",
 "resolution": "A2 moved, A3 waited"
}
```

6.3.2 Event Log Fields

Field	Description
time_step	Simulation time step when the event occurred.
event_type	Type of event recorded.
agent_id	Main agent involved in the event.
related_agents	Other agents involved, if relevant.
task_id	Related task, if relevant.
cell	Grid cell involved, if relevant.
details	Human-readable explanation.
resolution	How the event was handled, if relevant.

6.4 Event Types to Record

The system should record the following event types:

TASK_CREATED
TASK_ANNOUNCED
TASK_BID_SUBMITTED
TASK_ASSIGNED
TASK_PICKED_UP
TASK_COMPLETED
PATH_PLANNED
PATH_REPLANNED
RESERVATION_CREATED
RESERVATION_REJECTED
RESERVATION_RELEASED
MOVEMENT_INTENT_BROADCAST
AGENT_MOVED
AGENT_WAITED
INVALID_MOVE_PREVENTED
CONFLICT_DETECTED
CONFLICT_RESOLVED
DEADLOCK_DETECTED
DEADLOCK_RESOLUTION_STARTED
DEADLOCK_RESOLVED
CONGESTION_DETECTED
CONGESTION_AVOIDED
SIMULATION_STARTED
SIMULATION_PAUSED
SIMULATION_ENDED

6.5 Agent Decision Log

Each agent should also have a decision log. This is useful for explaining why an agent selected a task, waited, replanned, or changed route.

Agent Decision Log Schema

⟨/⟩ *JSON*

```
{
"time_step": 340,
"agent_id": "A4",
"state": "BIDDING",
"position": [6, 9],
"assigned_task": null,
"available_tasks": ["T7", "T8", "T9"],
"selected_action": "SUBMIT_BID",
"bid_task": "T8",
"bid_value": 18.6,
"reason": "lowest estimated cost based on distance, congestion, and workload"
}
```

Agent Decision Fields:

Field	Description
time_step	Current simulation time.
agent_id	Agent identifier.
state	Current agent state.
position	Agent grid position.
assigned_task	Current task ID, if assigned.
available_tasks	Visible tasks considered by the agent.
selected_action	Action selected by the agent.
bid_task	Task being bid on, if relevant.
bid_value	Bid cost submitted by the agent.
reason	Short explanation of the decision.

6.6 Metrics Collection

Metrics are collected at two levels:

1. **System-level metrics** — evaluate the whole warehouse.
2. **Agent-level metrics** — evaluate the behavior of each robot.

6.7 System-Level Metrics

6.7.1 Throughput

Throughput measures the number of completed tasks per simulation time.

$$\text{Throughput} = \frac{\text{N}_{\text{completed}}}{\text{T}}$$

Where:

- $N_{complete}$ is the number of complete tasks.
- T is the total simulation duration.

Higher throughput means the system completes more orders in less time.

6.7.2 Average Task Completion Time

This measures how long tasks take from creation to delivery.

$$AvgCompletionTime = avg(completion_{time_i} - arrival_{time_i})$$

Lower average completion time means tasks are being handled efficiently.

6.7.3 Collision Rate

Collision rate measures the number of collision events per simulation time.

$$\text{CollisionRate} = \frac{\text{N}_{\text{collision events}}}{\text{T}}$$

The target value is:

$$CollisionRate = 0$$

This is non-negotiable. If collision rate is greater than zero, the coordination strategy failed from a safety perspective.

6.7.4 Agent Utilization

Agent utilization measures how much of the simulation time agents spend actively working.

$$Utilization = average(\frac{active_{time_j}}{T})$$

Over all agents.

Where $active_{time_j}$ is the amount of time agent j spends moving, picking up, delivering, bidding, or replanning for a task.

High utilization is good only up to a point. If utilization is high because agents are constantly stuck, waiting, or congested, then the system is not actually efficient. That is why utilization must be interpreted together with throughput and completion time.

6.7.5 Path Efficiency

Path efficiency compares actual traveled distance with the shortest possible path in the static grid.

$$Path_{efficiency} = avg\left(\frac{optimal_{distance_i}}{actual_{distance_i}}\right)$$

Values closer to 1 mean the agent paths are close to optimal. Lower values indicate detours caused by congestion, reservations, conflicts, or poor planning.

6.7.6 Deadlock Resolution Time

Deadlock resolution time measures how long it takes to recover from a detected deadlock.

$$Deadlock_{Resolution\ time} = deadlock_{resolved\ time} - deadlock_{detected\ time}$$

For multiple deadlocks:

$$Avg(DeadlockResolutionTime) = average(resolution_{time_k})$$

The important validation condition is that this value must be finite. A deadlock that never resolves means the coordination mechanism failed.

6.7.7 Congestion Index

Congestion index measures how crowded shared bottleneck areas become.

$Congestion_{Index} = avg(number\ of\ agents\ in\ congested\ or\ bottleneck\ cells)$

A more precise implementation can use:

$$Congestion_{Index}(t) = sum\left(\frac{agents\ in\ region_r}{capacity_r}\right)$$

overall monitored regions.

Then average it over the full simulation.

These metric matters because zero collisions alone are not enough. A system can avoid collisions but still perform badly if agents constantly block each other

6.8 Agent-Level Metrics

Each agent should also be evaluated individually.

Metric	Meaning
tasks_completed_by_agent	Number of tasks completed by the agent.
active_time	Number of steps spent executing useful work.
idle_time	Number of steps spent with no task.
waiting_time	Number of steps spent waiting because of conflicts or reservations.
replanning_count	Number of times the agent had to replan.
distance_traveled	Total number of grid moves.
conflicts_involved	Number of conflicts involving the agent.
deadlocks_involved	Number of deadlocks involving the agent.
average_bid_value	Mean bid cost submitted by the agent.

These metrics help detect unfair or unbalanced behavior. For example, if one agent completes 80% of the tasks while others are idle, the auction strategy is probably biased or badly tuned.

6.9 Scenario-Level Analytics

Scenario	Main Metrics
Baseline	Throughput, AvgCompletionTime, CollisionRate, PathEfficiency
High Load	Throughput, AvgCompletionTime, Utilization, CongestionIndex
High Density	CollisionRate, CongestionIndex, PathEfficiency
Deadlock	DeadlockResolutionTime, CollisionRate
Dynamic Environment	Throughput, AvgCompletionTime, CongestionIndex
Scalability	Throughput, CongestionIndex, Utilization

Each validation scenario should generate a summary report.